



MOVIE RECOMMENDER SYSTEMS

 Submitted by: Asvithaa K

 College Name: Sri Shakthi Institute Of Engineering and Technology, Coimbatore

 Date: 16.03.2025

 Internship ID: UMIP277719

 Project Title: Movie Recommender Systems

 Domain: Data Science

 Tools used: ML, Python, Flask

Asvithaa

1. Introduction

In this project, we developed a **Movie Recommendation System** using **Collaborative Filtering** with the **Singular Value Decomposition (SVD)** technique. The goal of this system is to suggest personalized movie recommendations based on user preferences

2. Objectives of the Project

- ✓ Improve user experience by providing accurate and relevant movie recommendations.
- ✓ Reduce the effort required to find movies of interest.
- ✓ Utilize machine learning techniques to predict user preferences efficiently
- ✓ Implement a scalable and deployable recommendation model.

3. Tools & Technologies Used

- 📌 **Programming Language:** Python 🐍
- 📌 **Libraries Used:** pandas, numpy, surprise, flask
- 📌 **Model Used:** Singular Value Decomposition (SVD) 🤖
- 📌 **Deployment:** Flask

4. Dataset Preparation

Python code:

```
import pandas as pd

ratings = pd.read_csv('/content/ratings_small.csv')
movies = pd.read_csv('/content/movies_metadata.csv', low_memory=False)

print("Ratings Dataset:")
print(ratings.head())

print("\nMovies Dataset:")
print(movies.head())
```

Output:

 Ratings Dataset:

	userId	movieId	rating	timestamp
0	1	31	2.5	1260759144
1	1	1029	3.0	1260759179
2	1	1061	3.0	1260759182
3	1	1129	2.0	1260759185
4	1	1172	4.0	1260759205

Movies Dataset:

	adult	belongs_to_collection	budget	\
0	False	{'id': 10194, 'name': 'Toy Story Collection', ...}	30000000	
1	False	NaN	65000000	
2	False	{'id': 119050, 'name': 'Grumpy Old Men Collect...	0	
3	False	NaN	16000000	
4	False	{'id': 96871, 'name': 'Father of the Bride Col...	0	

genres \

0	[{'id': 16, 'name': 'Animation'}, {'id': 35, 'name': 'Comedy'}]
1	[{'id': 12, 'name': 'Adventure'}, {'id': 14, 'name': 'Fantasy'}]
2	[{'id': 10749, 'name': 'Romance'}, {'id': 35, 'name': 'Comedy'}]
3	[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'name': 'Drama'}]
4	[{'id': 35, 'name': 'Comedy'}]

	homepage	id	imdb_id	original_language	\
0	http://toystory.disney.com/toy-story	862	tt0114709	en	
1	NaN	8844	tt0113497	en	
2	NaN	15602	tt0113228	en	
3	NaN	31357	tt0114885	en	
4	NaN	11862	tt0113041	en	

5. Data Cleaning & Preprocessing

Clean the Data:

```
movies = movies[['id', 'title']]
movies = movies.rename(columns={'id': 'movieId'})

movies['movieId'] = pd.to_numeric(movies['movieId'], errors='coerce')
ratings['movieId'] = pd.to_numeric(ratings['movieId'], errors='coerce')

movies = movies.dropna()
ratings = ratings.dropna()

movies['movieId'] = movies['movieId'].astype(int)
ratings['movieId'] = ratings['movieId'].astype(int)

print("Cleaned Movies Dataset:")
print(movies.head())

print("\nCleaned Ratings Dataset:")
print(ratings.head())
```



Cleaned Movies Dataset:

	movieId	title
0	862	Toy Story
1	8844	Jumanji
2	15602	Grumpier Old Men
3	31357	Waiting to Exhale
4	11862	Father of the Bride Part II

Cleaned Ratings Dataset:

	userId	movieId	rating	timestamp
0	1	31	2.5	1260759144
1	1	1029	3.0	1260759179
2	1	1061	3.0	1260759182
3	1	1129	2.0	1260759185
4	1	1172	4.0	1260759205

6. Train the SVD Collaborative Filtering Model. 🚀

Load Data into Surprise Format:

```
from surprise import Dataset, Reader

reader = Reader(rating_scale=(0.5, 5))

data = Dataset.load_from_df(ratings[['userId', 'movieId', 'rating']],
                             reader)

print("Data successfully loaded into Surprise format!")
```

Train and Evaluate the Model:

```
from surprise import SVD
from surprise.model_selection import cross_validate

svd = SVD()

cross_validate(svd, data, measures=['RMSE', 'MAE'], cv=5, verbose=True)

trainset = data.build_full_trainset()
svd.fit(trainset)

print("Model training completed!")
```

➡ Evaluating RMSE, MAE of algorithm SVD on 5 split(s).

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std
RMSE (testset)	0.8916	0.9049	0.8943	0.8954	0.8940	0.8960	0.0046
MAE (testset)	0.6856	0.6954	0.6906	0.6909	0.6876	0.6900	0.0033
Fit time	1.30	1.66	1.66	1.30	1.30	1.44	0.18
Test time	0.11	0.15	0.21	0.09	0.18	0.15	0.04
Model training completed!							

7. Making Predictions & Recommendations

Predict a Rating for a Specific User & Movie

```
user_id = 1
movie_id = 31

prediction = svd.predict(user_id, movie_id)
print(f"Predicted rating for user {user_id} and movie {movie_id}:
{prediction.est:.2f}")
```

Output:

➡ Predicted rating for user 1 and movie 31: 2.21

Generate Movie Recommendations

```
import numpy as np

def recommend_movies(user_id, num_recommendations=10):
    movie_ids = movies['movieId'].unique()

    predictions = [svd.predict(user_id, movie_id).est for movie_id in
movie_ids]

    recommendations = pd.DataFrame({'movieId': movie_ids,
'predicted_rating': predictions})

    recommendations = recommendations.sort_values(by='predicted_rating',
ascending=False)
```

```

top_recommendations = recommendations.head(num_recommendations)

top_recommendations = top_recommendations.merge(movies, on='movieId')

return top_recommendations[['title', 'predicted_rating']]

recommended_movies = recommend_movies(1, 10)
print(recommended_movies)

```

Output:



	title	predicted_rating
0	While You Were Sleeping	3.696909
1	The Sicilian Clan	3.557517
2	The 39 Steps	3.510835
3	Amélie	3.486552
4	Nell	3.485810
5	Edward Scissorhands	3.477580
6	Lonely Hearts	3.476283
7	Hour of the Gun	3.475404
8	Broken Blossoms	3.474126
9	Aelita: Queen of Mars	3.455222

Saving our model to deploy in flask :

```

import pickle

with open('svd_model.pkl', 'wb') as f:
    pickle.dump(svd, f)

print("Model saved successfully!")

```

8. Deployment:

Creating app.py :

```
from flask import Flask, request, jsonify, render_template

import pandas as pd
import pickle
from surprise import Dataset, Reader

# Load trained SVD model
with open("svd_model.pkl", "rb") as model_file:
    svd = pickle.load(model_file)

# Load datasets
movies = pd.read_csv("movies_metadata.csv", low_memory=False)[["id", "title"]]
movies = movies.rename(columns={"id": "movieId"})
movies["movieId"] = pd.to_numeric(movies["movieId"],
errors="coerce").dropna().astype(int)

# Flask app
app = Flask(__name__)

# Serve the HTML page
@app.route("/")
def home():
    return render_template("index.html")

# Recommendation function
def recommend_movies(user_id, num_recommendations=10):
    movie_ids = movies["movieId"].unique()
    predictions = [svd.predict(user_id, movie_id).est for movie_id in movie_ids]
    recommendations = pd.DataFrame({"movieId": movie_ids, "predicted_rating":
predictions})
    recommendations = recommendations.sort_values(by="predicted_rating",
ascending=False)
    top_recommendations = recommendations.head(num_recommendations).merge(movies,
on="movieId")
    return top_recommendations[["title",
"predicted_rating"]].to_dict(orient="records")

# API endpoint
@app.route("/recommend", methods=["GET"])
def recommend():
```

```

user_id = int(request.args.get("user_id", 1))
num_recommendations = int(request.args.get("num_recommendations", 10))
recommendations = recommend_movies(user_id, num_recommendations)
return jsonify(recommendations)

if __name__ == "__main__":
    app.run(debug=True)

```

Creating html :

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Movie Recommendation System</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            margin: 50px;
        }
        input, button {
            padding: 10px;
            margin: 10px;
        }
        table {
            width: 50%;
            margin: auto;
            border-collapse: collapse;
        }
        th, td {
            border: 1px solid black;
            padding: 10px;
        }
    </style>
</head>
<body>

    <h1>Movie Recommendation System</h1>

    <label for="userId">Enter User ID:</label>
    <input type="number" id="userId" min="1" placeholder="Enter User ID">
    <button onclick="getRecommendations()">Get Recommendations</button>

```



```

<h2>Recommended Movies</h2>
<table id="moviesTable">
  <tr>
    <th>Movie Title</th>
    <th>Predicted Rating</th>
  </tr>
</table>

<script>
  function getRecommendations() {
    let userId = document.getElementById("userId").value;
    let url =
`http://127.0.0.1:5000/recommend?user_id=${userId}&num_recommendations=5`;

    fetch(url)
      .then(response => response.json())
      .then(data => {
        let table = document.getElementById("moviesTable");
        table.innerHTML = "<tr><th>Movie Title</th><th>Predicted
Rating</th></tr>";

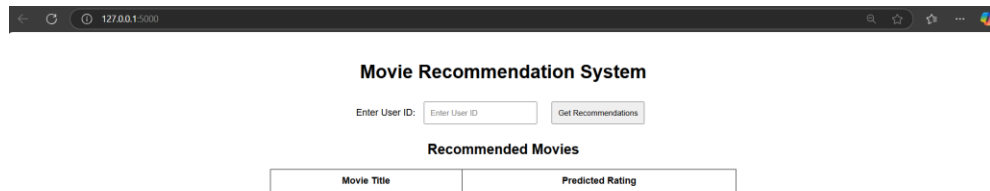
        data.forEach(movie => {
          let row = table.insertRow();
          let titleCell = row.insertCell(0);
          let ratingCell = row.insertCell(1);
          titleCell.innerHTML = movie.title;
          ratingCell.innerHTML = movie.predicted_rating.toFixed(2);
        });
      })
      .catch(error => console.log("Error fetching recommendations:",
error));
  }
</script>

</body>
</html>

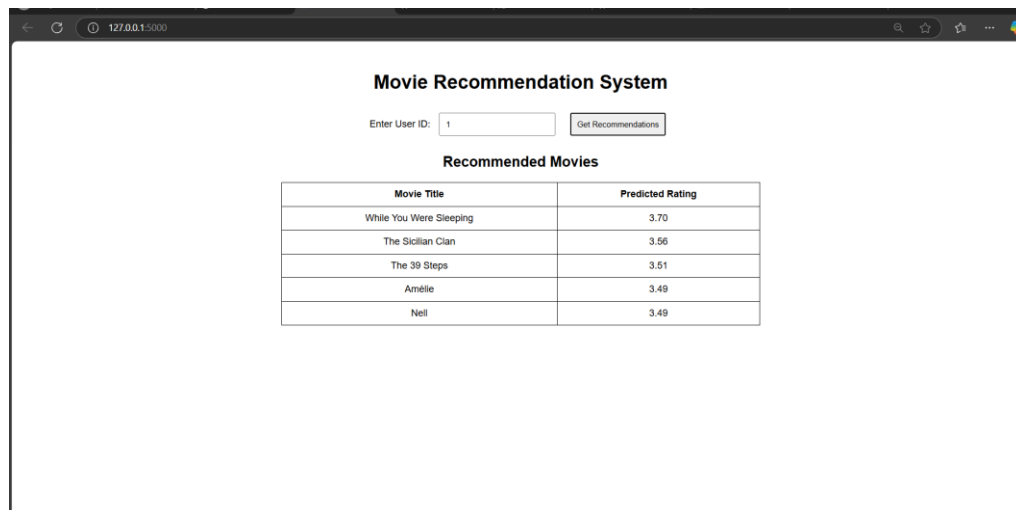
```

9. Final Output

Reference : Running on <http://127.0.0.1:5000>



The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000". The page title is "Movie Recommendation System". Below the title, there is a form with the label "Enter User ID:" followed by a text input field and a "Get Recommendations" button. Below the form, there is a section titled "Recommended Movies" which contains a table with two columns: "Movie Title" and "Predicted Rating".



The screenshot shows the same web browser window as above, but the "Enter User ID:" field now contains the value "1". The "Recommended Movies" table is populated with the following data:

Movie Title	Predicted Rating
While You Were Sleeping	3.70
The Sicilian Clan	3.56
The 39 Steps	3.51
Amélie	3.49
Neil	3.49

REFERENCE AND LINK :

GITHUB REPOSITORY LINK: <https://github.com/Asvithak07/movie-recommendation-system>

COLAB NOTEBOOK LINK :
<https://colab.research.google.com/drive/1DoQ0tdJxb3-jalUugJG-aRHNcYrT-njV?usp=sharing>

